

# Labels, paths and linearization of the $\lambda$ -calculus

Pedro Cunha      Sandra Alves      Mário Florido  
pedro.cunha@fc.up.pt    sandra@fc.up.pt    amflorid@fc.up.pt  
Departamento de Ciência de Computadores  
Faculdade de Ciências da Universidade do Porto  
Porto, Portugal

Linearization is essentially a program transformation in whose output all uses of input-bound variables correspond to different inputs, and which preserves the computational content of the original program. It can be syntactic, when applied to every subterm, or semantic, when applied only to functions which are used during reduction. One known example of semantic linearization is Weak Linearization, based on the maximal reduction strategy and which may not terminate when the program loops. The question we intend to answer here is this: how can we make such a linearization finer, so that we can get partial linearizations which work even in cases where the term is not strongly normalizing? In this paper we define a new linearization technique that answers the previous question using a linearization parametrized by a finite sequence of redexes, which is based on the idea of approaching the term as a tree.

## 1 Introduction: Linearization and its State-of-the-art

Linearization, also referred to as linear expansion (or just expansion, with an implicit linear qualification), is essentially a program transformation in whose output all uses of input-bound variables correspond to different inputs, and which preserves the computational content of the original program. In terms of the  $\lambda$ -calculus, this means all bound variables are bound by a distinct  $\lambda$ <sup>1</sup>. Thus, the term  $\delta = \lambda x.xx$  is not linear, whereas the term  $\lambda x_1 x_2.x_1 x_2$  is.

The first attempt at tackling this linearization problem was by Kfoury [7], which was characterized by his goal of “simulating the standard  $\lambda$ -calculus by a non-standard  $\lambda$ -calculus where we enforce the linearity condition on function evaluation”, which he would do by “[embedding] the standard  $\lambda$ -calculus  $\Lambda$  in a bigger calculus” that allows several arguments for an application and then restricting the  $\beta$ -reduction so that it can only take effect whenever the number of free occurrences of the applied function matches the number of arguments. In this work, an expanded term is precisely a term that has several arguments, as opposed to a standard  $\lambda$ -calculus term, which may only have one; Kfoury then shows how to expand a  $\lambda$ -term into a term of this expanded  $\lambda$ -calculus (Definition 2.16).

Following this, Florido and Damas defined the notion of linear expansion [6], whose first relevant difference from the previous is that it “stays” in the standard  $\lambda$ -calculus. Its basis are intersection types: given a term and its intersection type, this linearization obtains its (syntactical) linear form. This reveals an interesting aspect of linearization: since intersection types type exactly the strongly normalising (SN) terms, linear expansion is well-defined exactly for SN terms. So, for example, the linearization of  $\delta$  is  $\lambda x_1 x_2.x_1 x_2$ , whereas  $\Delta = \delta \delta$  cannot be linearized through this process, since it is not SN.

In this line, we also have the Weak Linearization of Alves and Florido [1], which becomes finer by linearizing only subterms that participate in some redex. This is done by utilizing legal paths, introduced

---

<sup>1</sup>More precisely, we can define *affine* functions, if we maintain that every  $\lambda$  binds **at most** one variable; or *strictly linear* functions, if we maintain that every  $\lambda$  binds **exactly one** variable. These are not equivalent; throughout this abstract, as in many previous works, we will use the first one.

by Asperti and Laneve [3] as a one-to-one correspondence with Lévy's redex families [9]; these are paths in the term when seen as a tree, which can be interpreted as computation paths. This further emphasizes the relation between computation and linearization; in fact, by Theorem 56 of [1], we know that when a term is not SN, it does not have an expansion. So, for example, the weak linearization of  $\delta$  is still  $\delta$ , since there is no redex (whence the “weak” linearization); however,  $\Delta$  and  $(\lambda xy.x)I\Delta$  have no weak linearization, since they are not SN. More interestingly, the weak linearization of  $(\lambda xy.xx)I(\delta I)$  would be  $(\lambda x_1x_2y.x_1x_2)II[(\lambda x_1x_2.x_1x_2)II]$ , since it works with *all* legal paths, i.e., all redexes (families).

The present work borrows some inspiration from previous works on linearization of the  $\lambda$ -calculus [1, 5, 6, 7, 10] and aims to contribute to this line of research providing a uniform framework for addressing linearization related problems.

## 2 Parametrized Linearization

The first linearization presented in the previous section is a syntactic linearization, in the sense that it linearizes *all* the non-linear subterms of the original term; it also depends on the typing of the term, whose inference algorithm is highly non-trivial (see, e.g., [4] and [8]).

Weak Linearization, on the other hand, relies on the *totality* of the legal paths of a term to terminate, and whenever there are infinite such paths, the linearization will not terminate (which is, again, the meaning of Theorem 56 of [1]). Besides this, since it works with all redexes, it will also expand terms that do not contribute to the normal form of the term, such as in the given example  $(\lambda xy.xx)I(\delta I)$ .

In the spirit of keeping in the  $\lambda$ -calculus and refining this process, the question we intend to answer is this: how can we refine such a linearization so that we can get partial linearizations (even when the term is not SN)? It seems reasonable to expect, given the strong relation with computation pointed at in the introduction, that there is a way in which we can do this, insofar as we regulate the interaction had with redexes; and so we answer thus: by a linearization parametrized by a (finite) sequence of redexes.

### 2.1 Redexes and Other Preliminary Definitions

Since we are parametrizing by a list of redexes, we must describe them in some (unique) way: this is the first challenge. In this matter, the work of Lévy, Asperti and Laneve is key ([9], [3]); we will particularly use labels. These are inductively defined as

$$\alpha := L \mid \bar{\alpha} \mid \underline{\alpha} \mid \alpha_1\alpha_2$$

where  $L$  is a set of atomic labels. We will use a specific type of atomic labels that depend on the node and on the tree; this labeling is done as follows (by “order”, we mean “order visited by depth-first search”):

- Mark all  $\lambda$  nodes with positive integers in increasing order, starting at 1;
- Mark all variables bound by some  $\lambda$  node  $n$  as  $n.k$ , where  $k$  is their order ( $n = 0$  for free variables);
- Mark all application nodes as  $M@N$ , where  $M$  is the label of the left subterm and  $N$  of the right.

A labeled term can be defined as  $\Lambda := x^\alpha \mid (\lambda x.\Lambda)^\alpha \mid (\Lambda\Lambda)^\alpha$ , where  $x$  is taken from a set of variables and  $\alpha$  is a label. These terms can be seen as trees when we consider variables as leaves,  $\lambda$ s as nodes with one child and applications ( $@$ ) as nodes with two children, in which each node<sup>2</sup> is annotated with its corresponding label (such as that in Fig. 2, left). The tree rewriting rule for reduction is in Fig. 1.

<sup>2</sup>In the cited works, only edges are labeled, but there is a direct bijection between the labeled edges and the labeled nodes.

Thus, since we can label each node as mentioned we can also associate computations in that term with specific labels (encoded by the reduction rule), which we can translate to paths in the original term-tree (Definition 5.1 of [3]). Since there is a correspondence between legal paths and these label yielded paths (as Theorem 8.11 of [3] asserts), working with either is equivalent when talking about redexes.

However, it must be noted that these uniquely represent redex *degrees*, which are unique *up to redex families* (Theorem 6.1 of [3]; p. 159 of [2]). In practice, this means that we do not have a guarantee of unicity of representations for every specific redex in an execution; e.g., the term  $T = (\lambda x.xx)(\lambda xy.(\lambda z.z)xy)$  would yield the labeled term in Fig. 2 after reducing its outermost redex.

Note that there are two specific redexes with degree 4. So, when we need to reference specific redexes in a reduction sequence, degrees, and thus labels and legal paths, are not enough.

Still, specific redexes of the same family arise only when they are copied in some reduction sequence; thus, since computations are “stored” in the tree’s nodes and structure, if we refer to the redexes as pairs of *the shortest paths from the root to the application and  $\lambda$  node* in an appropriate tree (i.e., obtained by a sequence of reductions from the original term, where the latter is a left subchild of the former), we have a unique characterization of such redexes.

Following are the required (preliminary) definitions and proposition;<sup>3</sup> unless specifically stated, by “term” we will always mean a labeled term.

**Proposition.** *In a term  $T'$ , obtained by reducing any number of redexes of a term  $T$ , no two different nodes have the same shortest path from the root.*

*Proof.* Suppose there were two nodes that had the same shortest path from the root,  $r_i$  and  $r_j$ . Since they must have the same labels (otherwise one of the nodes would have a label with an even number of atomic labels, which is not possible by Proposition 5.3 of [3]), these would have to be copies, since our initial labeling does not allow for repeated labels. This means that a redex, at some point, was reduced; thus, these copies are in place of the variables of the  $\lambda$  of the redex. But since there were (at least) two of these, we can assume, without loss of generality (if they are copies, we repeat the process for their origin), that they were not copies themselves, and thus must have different labels, say  $n.k$  and  $n.l$ . This,

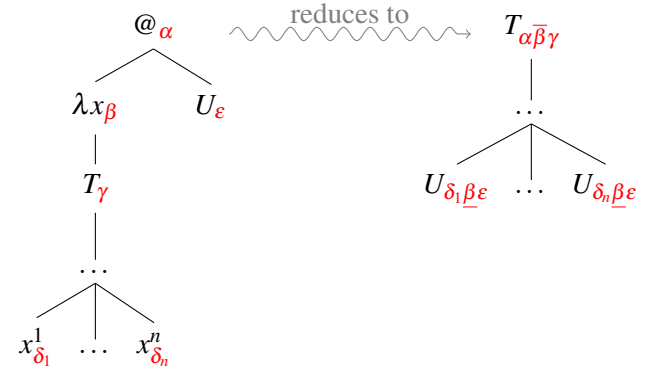


Figure 1: Labeled reduction rule. The degree of the redex is  $\beta$ .

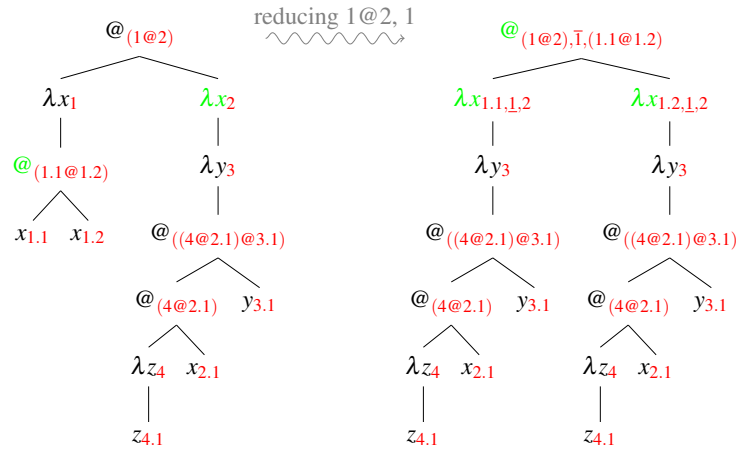


Figure 2:  $T$  after reducing leftmost-outermost redex, yielding  $(\lambda xy.(\lambda z.z)xy)(\lambda xy.(\lambda z.z)xy)$ .

<sup>3</sup>Proofs can be found in <https://github.com/pcunha1274/Proofs-for-TERMGRAPH>.

by the definition of labeled reduction, means that, somewhere in  $r_i$  and  $r_j$  we would have, respectively,  $n.k$  and  $n.l$ , which is a contradiction. Thus, there can be no two nodes with the same shortest path from the root.  $\square$

**Definition (Label Path).** *The label path of a node  $N$  in a term  $T$  is the (shortest) path from  $T$ 's root to  $N$ .*

**Definition (Direct Redex).** *A direct redex of a term  $T$  is a pair  $(r_@, r_\lambda)$ , where  $r_@, r_\lambda$  are the label paths of two nodes whose label ends in  $M@N, n$ , respectively, in  $T$  and  $n$  is the left subchild of  $M@N$ .*

**Definition (Virtual Redex).** *A virtual redex of a term  $T$  is a direct redex in a term  $T'$  obtained by reducing some (non-zero, finite) amount of direct redexes.*

**Definition (Redex Duality).** *For some redex  $r = (r_@, r_\lambda)$ , we say that  $r_@$  is the dual of  $r_\lambda$ , written  $d(r_\lambda)$ , and vice-versa.*

**Definition (List of Redexes).** *A list of redexes of a term  $T$  is a list of either direct or virtual redexes.*

The following sequentiality condition of a list of redexes guarantees that a redex list is sequential in the intuitive sense, by forcing the dependency of a redex on the previous ones.

**Definition (Sequential List of Redexes/Reduction Sequence).** *Let  $r_1, \dots, r_n$  be a list of redexes of a term  $T$ . We say it is sequential - or a reduction sequence - whenever each  $r_i$  is a direct redex of the term  $T'$  obtained by reducing  $r_1, \dots, r_{i-1}$ .*

Thus, we can talk about a sequence of redexes in a sense more general than that of a strategy of evaluation (as in, e.g., p. 83 of [1]); this will be our parameter.

For instance, in Fig. 2 the label path of the left child of the root is  $1@2, 1$ ; thus,  $r = (1@2; 1@2, 1)$  is a direct redex, which we will write  $(1@2 \mid 1)$ , since the path of the  $@$ -node is always a prefix of the path of the  $\lambda$ -node; and  $r_v = (1@2, \bar{1}, 1.1@1.2 \mid 1.1, \underline{1}, 2)$  is a virtual redex (note that it can be mapped in  $T$ ). Moreover, the list of redexes  $r, r_v$  is a reduction sequence, whereas  $r_v, r$  is not.

During the expansion, some nodes (along with their labels) will be copied, which will form clusters. So, when traversing these trees, clusters of  $\lambda$ -nodes are taken as one node and we specify which  $@$ -node we should go to with annotated paths.

**Definition (Cluster).** *A cluster of ( $\lambda$ - or  $@$ -)nodes in  $T$  is a set of nodes that form a subtree of  $T$  and share the same label.*

**Definition (Annotated path).** *An annotated path is a label path where application nodes that are followed by their right child in the path (i.e., that form the right edge of that application node) have annotations, more precisely, these nodes are pairs of the form  $(\ell, \pi \rightarrow n)$ , where  $\pi$  is some annotated path and  $n$  is a positive integer.*

**Definition (Multiplicity of a node/cluster).** *The multiplicity of a  $\lambda$ -node is the number of its bound variables; of a  $\lambda$ -cluster is the sum of the multiplicities of its  $\lambda$ -nodes; of a  $@$ -cluster is the number of nodes in it (i.e., the number of arguments).  $\mu(\pi)$  stands for the multiplicity of  $\pi$ , which is the path of a node (cluster).*

Thus, in Fig. 4 (left) we have a  $\lambda$ -node cluster with path  $1@3, 1$  and multiplicity 2 (examples of annotated paths can be found in the next subsection).

We now start the accessory definitions for our linearization function.

**Definition (Local interpretation of label paths).** *The local interpretation of a label  $\alpha$ , in a term  $T$ , with previous path  $p$  and list of redexes  $l_r$  is given by the function  $\text{interpret}_{\text{local}}$ , defined as follows:*

1.  $\text{interpret}_{\text{local}}(\alpha, T, p, l_r) = (T, p + \alpha, l_r)$ , with atomic  $\alpha$ ;

2.  $\text{interpret}_{\text{local}}(\bar{\alpha}, T, p, l_r) = (T, p + \bar{\alpha}, l_r)$ ;
3.  $\text{interpret}_{\text{local}}(\underline{\alpha}, T, p, l_r) =$ 
  - $(T, p + (\underline{\alpha}, d(\pi_{\lambda_n}) \rightarrow i), l_r)$ , if  $\alpha$  is atomic.  
*i is the position of n.k, which is the node that p leads to (by Proposition 5.3 of [3]), relative to the other bound variables of  $\pi_{\lambda_n}$  in T (in DFS order); the path  $\pi_{\lambda}$  is obtained by removing from p the sequence starting at n.k backwards until the first occurrence of n (exclusively), while its dual is found in  $l_r$ .*
  - $\text{interpret}_{\text{local}}((\alpha_3, d(\pi_{\lambda_n}) \rightarrow i), T, p + \underline{p'}, l'_r)$ , if  $\alpha = \alpha_1 \alpha_2 \alpha_3$ , with  $\alpha_1, \alpha_3$  atomic and  $\alpha_2$  non-atomic.  
 *$p'' = p' - p$  and  $(T, p', l'_r) = \text{interpret}_{\text{local}}(\alpha_2, \text{interpret}_{\text{local}}(\alpha_1, T, p, l_r))$ ; i and  $\pi_{\lambda}$  are obtained as in the previous case.*
4.  $\text{interpret}_{\text{local}}(\alpha_1 \alpha_2, T, p, l_r) = \text{interpret}_{\text{local}}(\alpha_2, \text{interpret}_{\text{local}}(T, \alpha_1, p, l_r))$ , where  $\alpha_1$  is atomic.

**Definition** (Interpretation (global)). *The (global) interpretation of a label path  $\alpha_1 \dots \alpha_n$ , in a term T, with previous path p and list of redexes  $l_r$  is given by the function  $\text{interpret}$ , defined as follows:*

$$\begin{aligned} \text{interpret}(\varepsilon, T, p, l_r) &= p \\ \text{interpret}(\alpha_1 \dots \alpha_n, T, p, l_r) &= \text{interpret}(\alpha_2 \dots \alpha_n, \text{interpret}_{\text{local}}(\alpha_1, T, p, l_r)) \end{aligned}$$

**Proposition** (Injectivity of  $\text{interpret}$ ).  *$\text{interpret}$  is injective over label paths.*

*Proof.* It suffices to show that local interpretation (i.e., interpretation of labels) is injective. Assume  $\alpha_1, \alpha_2$  are two labels, T is a term, p is some path and  $l_r$  a list of redexes; assume also that  $\text{interpret}(\alpha_1, T, p, l_r) = \text{interpret}(\alpha_2, T, p, l_r)$ . We proceed by induction on the structure of  $\alpha_1$ :

- If  $\alpha_1$  is an atomic label, then  $\text{interpret}(\alpha_1, T, p, l_r) = \alpha_1 = \text{interpret}(\alpha_2, T, p, l_r)$ , which, by definition, means that  $\alpha_1 = \alpha_2$ ;
- If  $\alpha_1 = \bar{\alpha}$ , then  $\text{interpret}(\bar{\alpha}, T, p, l_r) = (T, p + \bar{\alpha}, l_r) = (T, p + \alpha_2, l_r) = \text{interpret}(\alpha_2, T, p, l_r)$ , which means that  $\alpha_1 = \alpha_2$ ;
- If  $\alpha_1 = \underline{\alpha}$ , we proceed again by induction on the structure of  $\alpha$ :
  - if  $\alpha$  is atomic, then  $\text{interpret}(\underline{\alpha}, T, p, l_r) = (T, p + (\underline{\alpha}, d(\pi_{\alpha}) \rightarrow n), l_r) = \text{interpret}(\alpha_2, T, p, l_r)$ , which means that  $\alpha_1 = \alpha_2$ ;
  - if  $\alpha = \beta_1 \beta_2 \beta_3$  is non-atomic, then  $\text{interpret}(\alpha, T, p, l_r) = (T, p + \underline{p''} + (\beta_3, d(\pi_{\beta_3}) \rightarrow n), l'_r) = \text{interpret}(\alpha_2, T, p, l_r)$ , where  $p'' = p' - p$  and  $(T, p', l'_r) = \text{interpret}(\beta_2, \text{interpret}(\beta_1, T, p, l_r)) = \text{interpret}(\beta_2, T, p + \beta_1, l_r)$ . Our hypothesis implies that we also have  $\alpha_2 = \beta'_1 \beta'_2 \beta'_3$ , and thus, by the induction hypothesis we conclude that  $\text{interpret}(\beta'_2, T, p + \beta'_1, l_r) = (T, p', l'_r)$ . Thus,  $(T, p + \underline{p''} + (\beta_3, d(\pi_{\beta_3}) \rightarrow n), l'_r) = (\beta'_3, d(\pi_{\beta'_3}) \rightarrow n), l'_r)$  and thus  $\alpha_1 = \alpha_2$ .
- If  $\alpha_1 = \alpha \alpha'$ , with  $\alpha$  atomic and  $\alpha'$  non-atomic, then  $\text{interpret}(\alpha \alpha', T, p, l_r) = \text{interpret}(\alpha', \text{interpret}(\alpha, T, p, l_r)) = \text{interpret}(\alpha', T, p + \alpha, l_r)$ . Note that we must have  $\alpha_2 = \beta \beta'$  as well (otherwise we have a contradiction with our hypothesis); thus, we have, by the induction hypothesis, that  $\text{interpret}(\alpha', T, p + \alpha, l_r) = \text{interpret}(\beta', T, p + \beta, l_r)$ , which means that  $p + \alpha \alpha' = p + \beta \beta'$  and finally that  $\alpha_1 = \alpha_2$ .

Thus,  $\text{interpret}$  is injective. □

This means we lose no information regarding redexes when translating to annotated paths.

**Definition** (Local expansion). *The local expansion of a redex  $r'$  (expressed as an annotated path) in a term  $T$  is given by the function  $\text{expand}$ , which is defined as*

$$\text{expand}(r', T) = T'$$

where  $T'$  is obtained by the tree rewriting rule in Fig. 3.

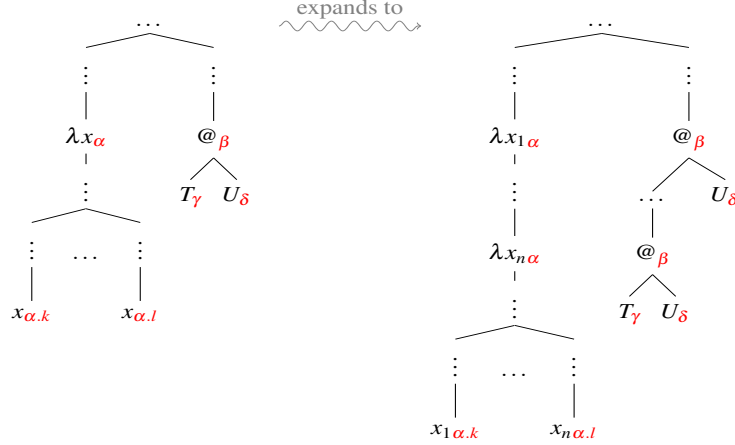


Figure 3: Tree-rewriting definition of  $\text{expand}$ .  $d(\lambda x_\alpha) = @_ \beta$ , which have paths  $r'_\lambda, r'_@$ ;  $n = \mu(\lambda x_\alpha)$ , and there are  $n$   $@_\beta$  nodes. Since they are analogous, we omit the other cases for space management reasons.

**Definition** (Balancing of a term). *Balancing of a term  $T$ , for a reduction sequence  $r_1, \dots, r_n$ , is given by the function  $\text{balance}$ , which is defined as:*

$$\text{balance}(T, r_1 \dots r_n) = \begin{cases} T & \text{if } \mu(r_{@i}) = \mu(r_{\lambda i}), i \in \{1, \dots, n\} \\ \text{expand}(r'_n, \dots \text{expand}(r'_1, T)) & \text{otherwise} \end{cases}$$

where  $r'_i = \text{interpret}(r_i, T, \varepsilon, l_{r_i})$  and  $l_{r_i} = (r'_1, \dots, r'_{i-1})$ .

Thus, we arrive at our main definition.

**Definition** ((Parametrized) Linearization). *Let  $T$  be a term and  $r_i, \dots, r_n$ , as well as  $l_r$  (which will be the list of evaluated redexes), a sequential list of redexes. The function  $\text{linearize}(r_i \dots r_n, T, l_r)$  is defined as*

$$\begin{aligned} \text{linearize}(\varepsilon, T, l_r) &= T \\ \text{linearize}(r_i \dots r_n, T, l_r) &= \text{linearize}(r_{i+1} \dots r_n, \text{balance}(\text{expand}(r'_i, T), l'_r), l'_r) \end{aligned}$$

where  $r'_i = \text{interpret}(r_i, T, \varepsilon, l_r)$ ,  $l'_r = l_r + r'_i$  and  $+$  is list concatenation.

The parametrized linearization of a term  $T$  with redex sequence  $r_1, \dots, r_n$  is  $\text{linearize}(r_1 \dots r_n, T, \varepsilon)$

**Conjecture** (Correctness).

Let  $T$  be a term,  $r_1, \dots, r_n$  a reduction sequence of  $T$  and  $E = \text{linearize}(r_1 \dots r_n, T, \varepsilon)$ . If  $T \rightarrow_{r_1} \dots \rightarrow_{r_n} T'$  and  $\text{balance}$  terminates, then there exists a reduction sequence of  $E$  such that  $E \rightarrow_{\beta}^* T'$ .

This conjecture would imply that, for a SN term  $T$  such that  $N = \text{nf}(T)$ , if  $\text{balance}$  terminates, then there exists a reduction sequence  $r_1, \dots, r_n$  such that  $E = \text{linearize}(r_1 \dots r_n, T, \varepsilon)$  and  $\text{nf}(E) = T'$ .

## 2.2 Examples

Consider the term  $2\ 2$ , where  $2$  is the Church numeral  $2 = \lambda f x. f(fx)$ . This is a SN term and we can normalize it in (at least) two different ways: by always reducing the leftmost, outermost redex or by reducing the rightmost, innermost one. Naturally, they will yield the same term (and thus, by the previous conjecture, will have the same final linearization when parametrized by one or the other), but their partial linearizations will not be equal.

For the leftmost-outermost strategy, consider only the first two redexes,  $r_1 = (1@3 \mid 1)$  and  $r_2 = (1@3, \bar{1}, 2, 1.1@(1.2@2.1) \mid 1.1, \underline{1}, 3)$ .

The iteration for  $r_1$  is  $\text{balance}(\text{expand}(r_1, T), [r_1]) = T'$ , which is represented in Fig. 4 (left; for space management reasons, single nodes with multiple arguments represent clusters of @-nodes).

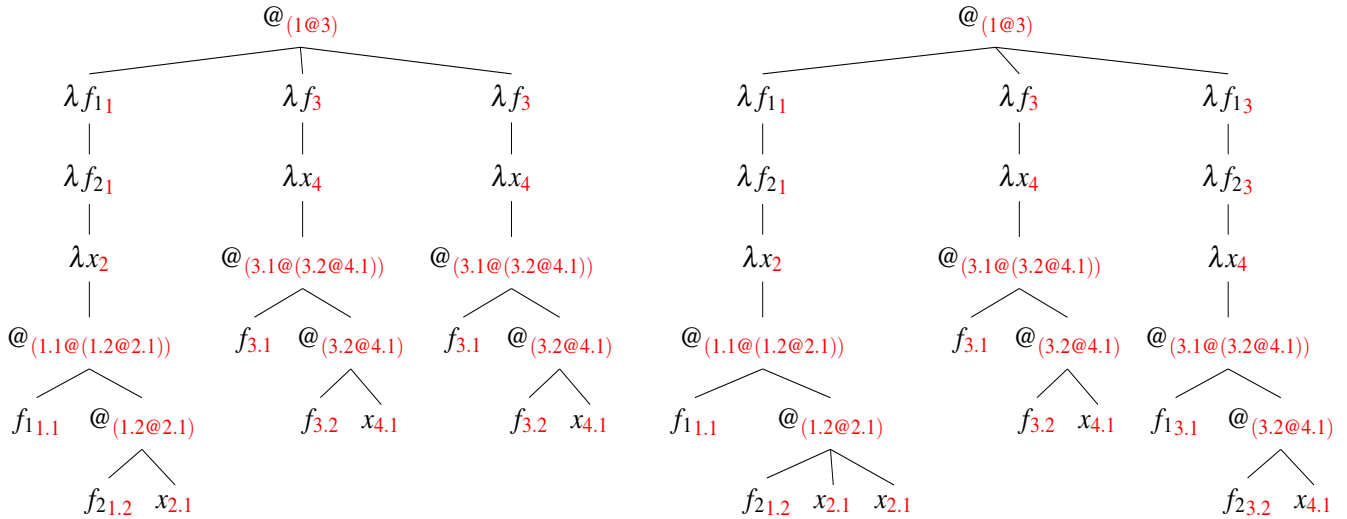


Figure 4: Left:  $T$  after expanding  $r_1$ . Right: Expanding the next rightmost-innermost redex of  $T$ .

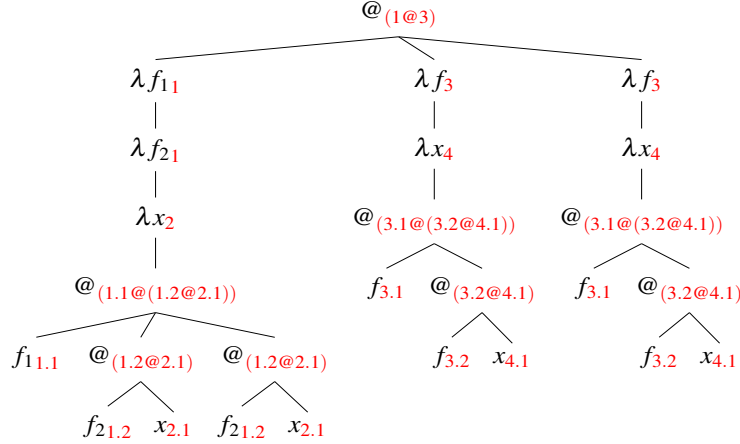
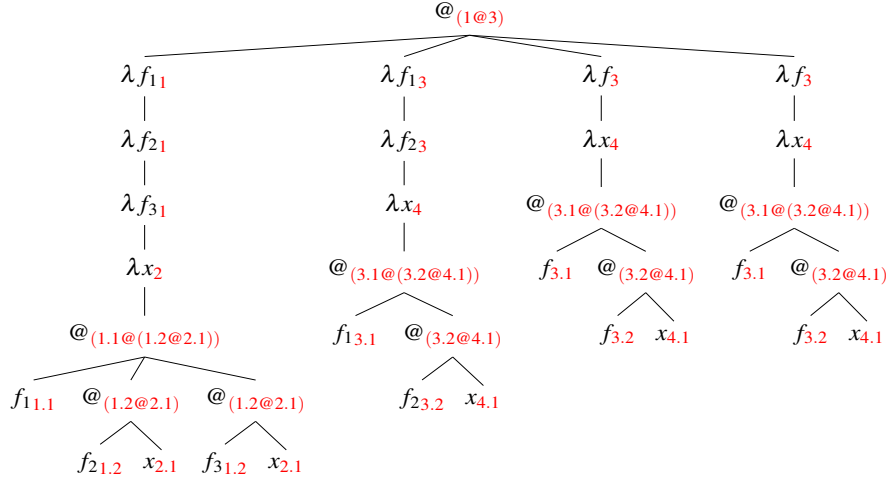
For  $r_2$ , we get  $\text{balance}(\text{expand}(r'_2, T'), l_r)$ , with  $l_r = [(1@3 \mid 1), r'_2]$ , which yields:

1.  $r'_2 = \text{interpret}([1@3, \bar{1}, 2; 1.1@(1.2@2.1) \mid 1.1, \underline{1}, 3], T', \varepsilon, l_r) = 1@3, \bar{1}, 2; 1.1@(1.2@2.1) \mid 1.1, (1, 1@3 \rightarrow 1), 3 = \pi_{@2} \mid 1.1, (1, 1@3 \rightarrow 1), 3$
2.  $\text{expand}(r'_2, T') = T''$ , where  $T''$  is as in Fig. 5.
3.  $\text{balance}(T'', [(1@3 \mid 1), (\pi_{@2} \mid 1.1, (1, 1@3 \rightarrow 1), 3)]) = \text{balance}(\text{expand}(r'_2, \text{expand}(r'_1, T'')), [r'_1, r'_2]) = \text{balance}(\text{expand}(r'_2, T'''), [r'_1, r'_2]) = \text{balance}(T''''', [r'_1, r'_2]) = T'''''$ , where  $T'''''$  is as in Fig. 6.

Note that we have  $2\ 2 \rightarrow_{r_1, r_2}^* \lambda f x. (2f)[(2f)x]$  and that the term in Fig. 6 also reduces to this term.

For the first two redexes of the rightmost-innermost strategy ( $r_1$  is also the first), we would have, instead, the expanded term in Fig. 4 (on the right). However, the two expansions would yield the same “final” result.

Another interesting example is whenever there are redexes that do not matter for the normal form: e.g., the normal form of  $(\lambda xy.xx)I(\delta I)$  is  $I = \lambda x.x$ , and the inner redex  $\delta I$  is, in a sense, irrelevant to it. However, in this case, standard weak linearization will give  $(\lambda x_1 x_2 y. x_1 x_2)II[(\lambda x_1 x_2. x_1 x_2)II]$ , which would be the same output if our linearization was parametrized by all the possible redexes. Still, if we

Figure 5:  $T''$ : 2nd iteration, after local expansion, before balancingFigure 6:  $T'''$ : 2nd iteration, after local expansion and balancing

started with  $(1@(3@4) \mid 1)$  (the leftmost-outermost redex) and never reduced the inner redex, we would still get the normal form and would have the expansion  $(\lambda x_1 x_2 y. x_1 x_2)II[\delta I]$ . This can only happen for terms with nodes that erase subterms (i.e., with multiplicity 0).

Notably, it is also possible to expand  $\Delta$ , or any other non-SN term, by using a finite list of redexes.

### 3 Ongoing/Future Work

We are finishing the proof of correctness for the linearization; the key lemma is that  $\text{expand}(r, T)$  gives a term that is linear for  $r$ . The termination of  $\text{balance}$  makes sure no relevant term remains “unbalanced” after successive applications of  $\text{expand}$ . Next, we intend on establishing that the presented linearization, when parametrized by the maximal strategy, is equivalent to the Weak Linearization of [1]. Following this, it is our goal to prove that  $\text{balance}$  terminates. If we succeed, with the help of the previous results, we can further the study of the conjecture in [1], the converse of Theorem 56 (if a term

is SN, then it has a weak linearization). From here, the natural way is seeing how this relates to syntactical linearization, e.g., whether it is possible to change the procedure to match the syntactical linearization and what is required; relations with other linearizations, specifically with the Taylor expansion [5], should also be convenient to explore at this stage. Finally, the strong match between syntactical linearization and intersection types, as well as the granularity and parametric character of the presented linearization, prompts us to look at possible relations with Carlier’s work [4]: a type inference algorithm that gradually infers the intersection type of a term based on the evaluation of redexes.

## References

- [1] S. Alves and M. Florido. Weak linearization of the lambda calculus. *Theoretical Computer Science*, 342(1):79–103, Sept. 2005.
- [2] A. Asperti and S. Guerrini. *The optimal implementation of functional programming languages*. Cambridge University Press, 01 1998.
- [3] A. Asperti and C. Laneve. Paths, computations and labels in the lambda-calculus. *Theoretical Computer Science*, 142(2):277–297, May 1995.
- [4] S. Carlier and J. B. Wells. Type inference with expansion variables and intersection types in system E and an exact correspondence with beta-reduction. In *Proceedings of the 6th ACM SIGPLAN international conference on Principles and practice of declarative programming*, pages 132–143, Verona Italy, Aug. 2004. ACM.
- [5] T. Ehrhard and L. Regnier. Uniformity and the Taylor expansion of ordinary lambda-terms. *Theoretical Computer Science*, 403(2-3):347–372, Aug. 2008.
- [6] M. Florido and L. Damas. Linearization of the lambda-calculus and its relation with intersection type systems. *Journal of Functional Programming*, 14(5):519–546, Sept. 2004.
- [7] A. J. Kfoury. A linearization of the lambda-calculus and consequences. *Journal of Logic & Computation*, 10(3), 2000.
- [8] A. J. Kfoury and J. B. Wells. Principality and decidable type inference for finite-rank intersection types. In *Proceedings of the 26th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 161–174, San Antonio Texas USA, Jan. 1999. ACM.
- [9] J.-J. Levy. Optimal reductions in the lambda calculus. *To H.B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, 01 1980.
- [10] D. Mazza, L. Pellissier, and P. Vial. Polyadic approximations, fibrations and intersection types. *Proceedings of the ACM on Programming Languages*, 2(POPL):1–28, Jan. 2018.